

# ADS Project Report

**Name:** Vandan Agrawal

**UFID:** 20175452

**E-mail:** [vandan.agrawal@ufl.edu](mailto:vandan.agrawal@ufl.edu)

## Classes and functions:

- 1) **class order(orderID, currentTime, orderValue, deliveryTime, eta, priority):** Class to create an object of an order which stores its id, The time it was created, its orderValue, its deliveryTime, its eta, and its priority, respectively.
- 2) **class pTreeNode(priority, orderId):** Node of Priority AVL tree. It stores the priority and a set 'orders' which stores all orderIds of orders with the given priority. It also stores address of left and right children, and its own height. Initial values:

```
self.priority = val
self.orders=set([orderId])
self.left = None
self.right = None
self.height = 1
```

- 3) **class eTreeNode(ETA, orderId):** Node of Priority AVL tree. It stores the ETA and an orderId for corresponding ETA. It also stores address of left and right children, and its own height. Initial values:

```
self.ETA = ETA
self.orderId=orderId
self.left = None
self.right = None
self.height = 1
```

- 4) **class Priority\_Tree:** Priority AVL tree.
  - a. **insert(self, root, priority, orderId):** Function to insert a new priority node in priority tree 'root', with priority 'priority' and order id 'orderId'. Calls functions to balance the tree at 'root'. Returns new root of the tree.  
Complexity:  $O(\log(n))$
  - b. **delSmallest(self, root):** Delete the smallest priority node in a given Priority tree 'root' Self balances at 'root' and has complexity  $O(\log(n))$ . Returns new root of the tree.
  - c. **delete(self, root, orderId):** Deletes order of 'orderId' from priority tree 'root'. If the node has no more orders left, it deletes the node, and calls delSmallest(root.right) in the process. Returns new root of the tree.  
Complexity:  $O(\log(n))$

- d. **searchMin(self, root, priority):** Returns a priority tree node in priority tree 'root' which has the smallest in all the priorities greater than or equal to the given 'priority'. If none exists, returns inf and 'None'. Complexity:  $O(\log(n))$
- e. **update\_ETAs(self, root, priority, uptime, inprocess=None):** Updates all ETAs of orders who have a priority less than given 'priority' in priority tree 'root' with given 'uptime' except for order Id given 'inprocess', if any order is currently being delivered. This function also used delete and insert functions in ETA\_Tree. Returns list of all updated orders. Complexity  $O(n\log(n))$
- f. **leftRotate(self, x):** Performs a left AVL tree rotation on tree node 'x' and returns new root. Complexity:  $O(1)$
- g. **rightRotate(self, x):** Performs a left AVL tree rotation on tree node 'x' and returns new root. Complexity:  $O(1)$
- h. **getHeight(self, root):** Returns height of tree 'root' if it exists else returns 0. Complexity:  $O(1)$
- i. **getBalance(self, root):** Returns balance factor of tree 'root' if it exists else returns 0. Complexity:  $O(1)$

5) **class ETA\_Tree:** ETA AVL tree.

- a. **insert(self, root, eta, orderId):** Function to insert a new ETA tree node in ETA tree 'root', with ETA 'ETA and order id 'orderId'. Calls functions to balance the tree at 'root'. Returns new root of the tree. Complexity:  $O(\log(n))$
- b. **delSmallest(self, root):** Delete the smallest ETA node in a given ETA tree 'root' and Self balances at 'root' and has complexity  $O(\log(n))$ . Returns new root of the tree.
- c. **delete(self, root, orderId):** Deletes ETA node with order 'orderId' in tree 'root'. Returns new root of the tree. Complexity:  $O(\log(n))$
- d. **getRank(self, root, eta):** Recursively checks and eventually returns the number of orders that need to be completed by time 'eta' in the ETA tree 'root'. Complexity:  $O(n)$
- e. **searchMin(self, root, eta):** Returns ETA and ETA tree node in ETA tree 'root' which has the smallest in all the ETAs greater than the given 'ETA. If none exist, returns inf and 'none'. Complexity:  $O(\log(n))$
- f. **searchMax(self, root, eta):** Returns ETA and ETA tree node in eta tree 'root' which has the largest in all the ETAs lesser than or equal to the given 'ETA. If none exists, then returns -inf and 'None'. Complexity:  $O(\log(n))$
- g. **deliverTill(self, root, eta):** Delivers, prints, and deletes all the orders till 'eta'. Also removes them from the dictionary of orders and the priority tree. This function calls Priority\_Tree.delete(root.orderId). Returns new root of the tree. Complexity:  $O(n(\log(n)))$
- h. **leftRotate(self, x):** Performs a left AVL tree rotation on tree node 'x' and returns new root. Complexity:  $O(1)$
- i. **rightRotate(self, x):** Performs a left AVL tree rotation on tree node 'x' and returns new root. Complexity:  $O(1)$
- j. **getHeight(self, root):** Returns height of tree 'root' if it exists else returns 0. Complexity:  $O(1)$
- k. **getBalance(self, root):** Returns balance factor of tree 'root' if it exists else returns 0. Complexity:  $O(1)$

## Global Variables:

**ET:** ETA tree object to call its functions. Initialised **ETA\_Tree()**

**PT:** Priority tree object to call its functions. Initialised **Priority\_Tree()**

**myETAtree:** Stores root node of ETA tree used. Initialized **None**

**myPriorityTree:** Stores root node of Priority tree used. Initialized **None**

**orders\_dict:** Dictionary (Hashmap) to store all objects of Order

**output:** Stores the lines that are to be outputted in output file.

## Driver functions:

### 1) *Print(par1, par2=None):*

- a. Prints the order details of the given 'orderId' when only par1 is given. Prints error when no such order is present. Complexity:  $O(1)$
- b. Prints all the orders that will be delivered within the given times (including both times) and are undelivered. If order list is empty, prints "There are no orders in that time period". Complexity:  $O(n \log(n))$

### 2) *Quit():*

Calls `ETA_Tree.deliverTill` for Infinity. In other words, delivers all remaining orders. Complexity:  $O(n \log(n))$

*All other functions used have the same name, prototype, and work just as mentioned in the requirements.*

## Libraries used:

- a. **math:** To get `math.inf` for `searchMin` and `searchMax` in both trees.
- b. **sys:** To change output file from IPython console to output file.

## Driver Code:

All global variables and classes mentioned above are declared and defined in the driver. All functions that are to be passed as commands are also defined directly in the driver, NOT in any class. All functions use ET and PT to access the **tree functions** (that is, they are not accessed statically), and `myETAtree` and `myPriorityTree` to access the **ROOT NODES** of their respective trees.

Inputs input file and stores it in a variable. Changes 'print' to 'Print' since 'print' is already a necessary function in Python. Stores all the functions in list 'commands' and runs function 'eval' to evaluate them. Also closes both the input, and output file.